BỘ KHOA HỌC VÀ CÔNG NGHỆ

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

HỆ THỐNG BÁO CÁO & ĐIỀU PHỐI SỰ CỐ TRƯỜNG HỌC

SCHOOL INCIDENT DISPATCH SYSTEM

Giảng viên hướng dẫn:	Msc. Ngô Văn Quyền
Danh sách thành viên:	Đào Duy Hải - B23DCCN268
	Phan Thanh Bình - B23DCCN086
	Đỗ Hữu Nam – B23DCCN575
Nhóm thực hiện:	Nhóm 11
Học phần:	Lập trình mạng
Năm học:	2026-2027

Hà Nội, năm 2026

Nhận xét của giảng viên chấm bài:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm số: .....

Điểm chữ: .....

Giảng viên *(ký và ghi rõ họ tên)*: .....

LỜI CAM ĐOAN

Chúng em xin cam đoan đề tài "Hệ thống Báo cáo & Điều phối Sự cố Trường học (School Incident Dispatch System – SIDS)" là công trình nghiên cứu và phát triển phần mềm độc lập của tập thể Nhóm 11 dưới sự hướng dẫn của Giảng viên phụ trách học phần Lập trình Mạng.

Toàn bộ các nội dung phân tích bài toán, thiết kế kiến trúc phân tầng, đặc tả giao thức mạng đóng khung nhị phân (Length-Prefixed Binary Protocol), các thuật toán điều phối đa luồng, xử lý tương tranh nguyên tử (Atomic Claim) và toàn bộ các số liệu kiểm nghiệm hiệu năng trong báo cáo này đều được đo đạc trực tiếp từ các thực nghiệm thực tế giữa hai máy tính vật lý độc lập qua môi trường mạng riêng ảo Radmin VPN, tuyệt đối không sao chép nguyên mẫu hay sử dụng số liệu giả lập từ bất kỳ nghiên cứu nào khác. Các tài liệu, công cụ và thư viện mã nguồn mở được tham khảo trong đề tài đều có trích dẫn nguồn gốc rõ ràng, hợp lệ theo quy chuẩn học thuật.

Hà Nội, ngày 31 tháng 10 năm 2026

Đại diện Nhóm thực hiện

(Ký và ghi rõ họ tên)

Đào Duy Hải

LỜI CẢM ƠN

Để hoàn thành đề tài bài tập lớn môn học Lập trình Mạng này, nhóm chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất tới Ban Giám hiệu, Khoa Công nghệ Thông tin cùng toàn thể các thầy cô giáo đã tận tình giảng dạy, truyền đạt những kiến thức nền tảng quý báu về kiến trúc mạng máy tính, mô hình truyền thông tầng Transport (TCP/UDP), cơ chế dồn/phân kênh, đồng bộ hóa đa luồng và các nguyên lý thiết kế ứng dụng phân tán.

Đặc biệt, nhóm xin gửi lời tri ân sâu sắc tới Thầy giáo hướng dẫn học phần Lập trình Mạng. Những lời nhận xét, góp ý chuyên môn khắt khe và định hướng sát sao của Thầy về việc thể hiện rõ bản chất lập trình mạng (Network Programming), cơ chế truyền nhận file nhị phân qua Socket, phân tích điểm gãy của hệ thống và giải quyết xung đột dữ liệu đa người dùng đã giúp nhóm chúng em không chỉ hoàn thiện phần mềm theo đúng tiêu chuẩn kỹ thuật chuyên nghiệp mà còn trưởng thành rõ rệt trong tư duy thiết kế hệ thống mạng thực tế.

Mặc dù nhóm đã nỗ lực hết mình với tinh thần làm việc nghiêm túc, đề tài chắc chắn khó tránh khỏi những thiếu sót nhất định do giới hạn về thời gian và kinh nghiệm thực tiễn. Nhóm chúng em rất mong tiếp tục nhận được những ý kiến đóng góp, chỉ dẫn quý báu từ Thầy cô để sản phẩm ngày càng hoàn thiện hơn và có khả năng áp dụng vào thực tế quản lý trường học trong tương lai.

Nhóm nghiên cứu xin chân thành cảm ơn!

MỤC LỤC

LỜI CAM ĐOAN **3**
LỜI CẢM ƠN **4**
MỤC LỤC **5**
DANH MỤC HÌNH ẢNH **7**
DANH MỤC BẢNG BIỂU **8**
TÓM TẮT **9**
HỆ THỐNG BÁO CÁO & ĐIỀU PHỐI SỰ CỐ TRƯỜNG HỌC (SCHOOL INCIDENT DISPATCH SYSTEM – SIDS) **11**
I. GIỚI THIỆU **11**
1.1. Bối cảnh và lý do chọn đề tài 11
1.2. Mục tiêu và phạm vi đề tài 11
1.3. Giới thiệu sơ lược phương pháp thực hiện 13
II. KHẢO SÁT CÁC HỆ THỐNG HIỆN NAY **15**
2.1. Kênh tiếp nhận thủ công và nhóm mạng xã hội (Hotline, Zalo, Forms) 15
2.2. Hệ thống Service Desk / IT Helpdesk thương mại (Jira, Freshdesk) 15
2.3. Hệ thống phản ánh hiện trường đô thị thông minh (Hệ thống 1022) 15
2.4. Thảo luận và hướng giải quyết cải tiến của SIDS 15
2.5. Phát biểu bài toán (Yêu cầu chức năng, phi chức năng, Input/Output) 17
III. PHƯƠNG PHÁP THỰC HIỆN (METHODOLOGY) **20**
3.1. Sơ đồ Use Case tổng thể 20
3.2. Kiến trúc tổng thể và vai trò các thành phần 20
3.3. Chương trình được xây dựng 22
IV. KIỂM NGHIỆM HỆ THỐNG **27**
4.1. Các kịch bản kiểm nghiệm hiệu năng 27
4.2. Tiến hành và ghi nhận kết quả kiểm nghiệm 28
V. THẢO LUẬN **32**
5.1. Phân tích sâu về hệ thống mạng 32
5.2. Đánh giá hệ thống qua các thực nghiệm 33
VI. KẾT QUẢ, HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN **35**

6.1. Kết quả đạt được 35
6.2. Hạn chế của hệ thống 35
6.3. Hướng phát triển mở rộng trong thực tế 36
VII. KẾT LUẬN 38
TÀI LIỆU THAM KHẢO 39
PHỤ LỤC 40
Phụ lục A. Bảng tra cứu toàn bộ OpCode giao thức 0xCAFE. 40

DANH MỤC HÌNH ẢNH

Hình 3.1. Sơ đồ Use Case tổng thể hệ thống điều phối sự cố trường học SIDS.....20

Hình 3.2. Sơ đồ kiến trúc phân tầng hệ thống Client-Server TCP song công. 21

Hình 3.3. Cấu trúc khung gói tin nhị phân chuẩn Length-Prefixed (Magic 0xCAFE). 22

Hình 3.4. Sơ đồ tuần tự (Sequence Diagram) luồng báo cáo và đẩy thông báo tự động. 23

Hình 3.5. Cơ chế tranh chấp tiếp nhận nguyên tử (Atomic Claim Transaction). 25

Hình 3.6. Luồng truyền nhận file nhị phân qua Socket TCP (TYPE_GET_IMAGE). 26

Hình 4.1. Mô hình môi trường kiểm thử phân tán qua mạng riêng ảo Radmin VPN. 27

Hình 4.2. Biểu đồ tỷ lệ kết nối thành công theo số lượng Client trực tuyến đồng thời. 29

Hình 4.3. Biểu đồ tương quan giữa số lượng Client, độ trễ phản hồi và thông lượng gửi ảnh.....30

DANH MỤC BẢNG BIỂU

Bảng 2.1. So sánh đối chiếu kỹ thuật giữa SIDS và các giải pháp hiện nay. 16

Bảng 2.2. Đặc tả các luồng Dữ liệu Đầu vào (Input) và Đầu ra (Output) cốt lõi. 19

Bảng 3.1. Đặc tả định dạng khung nhị phân giao thức 0xCAFE. 23

Bảng 3.2. Danh mục sự cố và phòng ban phụ trách.....24

Bảng 4.1. Thông số phần cứng và môi trường mạng thực nghiệm. 27

Bảng 4.2. Kết quả Kịch bản 1 – Khảo sát Sức chứa Kết nối Đồng thời (Stress Limit) 28

Bảng 4.3. Kết quả Kịch bản 2 – Đo lường Áp lực Tải Cao & Băng thông.(Load Test kèm Ảnh 250KB) (Mỗi Client liên tục gửi báo cáo sự cố kèm tệp tin ảnh JPEG dung lượng 250 KB qua mạng VPN).....29

Bảng 4.4. Kết quả Kịch bản 3 – Kiểm Thử Tranh Chấp Nhận Việc Đồng Thời (Race Condition & ACID) (Ép 10 kỹ thuật viên đồng thời bấm nhận đúng 1 sự cố khẩn cấp bằng CountDownLatch(10)) 30

Bảng 4.5. Kết quả Kịch bản 4 – Áp lực Bão Tin Nhắn Chat Thời Gian Thực (Chat Stress) (Bắn tải 20 luồng đồng thời × 15 tin nhắn = 300 tin nhắn dồn dập vào 1 phòng chat).....31

Bảng 6.1. Bảng đối chiếu kết quả thực nghiệm với các chỉ tiêu thiết kế ban đầu. ...35

TÓM TẮT

Trong công tác quản lý vận hành cơ sở vật chất và an toàn học đường tại các trường đại học, cao đẳng hiện nay, việc phát hiện, tiếp nhận và điều phối xử lý các sự cố khẩn cấp (như chập cháy điện, vỡ ống nước, kẹt thang máy, hỏng thiết bị thí nghiệm, ngất xỉu y tế hay gây rối an ninh) vẫn phụ thuộc chủ yếu vào các kênh liên lạc phân tán như gọi điện thoại trực tiếp, giấy báo hỏng hóc hoặc các nhóm mạng xã hội (Zalo, Messenger). Những phương thức này bộc lộ nhiều điểm nghẽn nghiêm trọng: độ trễ tiếp nhận cao, thất lạc thông tin, không phân định trách nhiệm rõ ràng, dễ xảy ra tranh chấp nhận việc trùng lặp (Double-booking) và thiếu cơ chế giám sát kiểm toán tập trung.

Đề tài "Hệ thống Báo cáo & Điều phối Sự cố Trường học (School Incident Dispatch System – SIDS)" được nghiên cứu và phát triển nhằm giải quyết triệt để bài toán trên thông qua việc vận dụng các nguyên lý cốt lõi của môn học Lập trình Mạng. Hệ thống được xây dựng trên nền tảng Java 17 LTS, áp dụng mô hình Client/Server tập trung kết hợp truyền thông song công (Full-Duplex TCP Streaming) qua Socket thuần (Raw Socket) mà không phụ thuộc vào các Web Framework cồng kềnh. Đề tài tự thiết kế một giao thức đóng khung nhị phân chuẩn Length-Prefixed Framing (Magic Header: 0xCAFE), cho phép ghép chung tiêu đề văn bản UTF-8 và tệp tin ảnh hiện trường nhị phân nguyên bản trong một luồng TCP duy nhất mà không qua chuyển đổi Base64. Phía Server tích hợp kiến trúc đa luồng Worker Thread Pool, cơ chế lưu trữ SQLite ở chế độ ghi nhật ký phía trước (WAL Mode) và thuật toán giải quyết tranh chấp nguyên tử (Atomic Claim), đảm bảo tuyệt đối không xảy ra tình trạng nhiều cán bộ cùng nhận trùng một sự cố.

Kết quả kiểm nghiệm thực tế giữa hai máy tính vật lý độc lập qua môi trường mạng diện rộng Radmin VPN cho thấy: Hệ thống duy trì ổn định 100% kết nối đồng thời ở quy mô 700 clients trên máy chủ cấu hình khiêm tốn (Intel Core i3, RAM 4GB); xử lý tải gửi ảnh hiện trường 250 KB dồn dập từ 100 clients với tỷ lệ thành công đạt 99.5% (băng thông mạng đạt đỉnh 2.61 MB/s); thông lượng chuyển tiếp tin nhắn chat

thời gian thực đạt trung bình 7.537 msg/s; và giải quyết tranh chấp đồng thời đạt 0.0% Deadlock và 0.0% Double-booking. Hệ thống chứng minh tính khả thi cao, tiết kiệm tài nguyên và sẵn sàng triển khai thực tế.

Từ khóa: Lập trình Mạng, Raw TCP Socket, Giao thức nhị phân Length-Prefixed, Điều phối sự cố, Đa luồng (Multithreading), Atomic Claim, SQLite WAL, JavaFX.

HỆ THỐNG BÁO CÁO & ĐIỀU PHỐI SỰ CỐ TRƯỜNG HỌC (SCHOOL INCIDENT DISPATCH SYSTEM – SIDS)

I. GIỚI THIỆU

1.1. Bối cảnh và lý do chọn đề tài

Trong môi trường giáo dục hiện đại, quy mô cơ sở hạ tầng ngày càng lớn với hàng chục dãy nhà, giảng đường, phòng thí nghiệm, khu thể thao và ký túc xá, nên các sự cố học đường phát sinh thường xuyên và đa dạng: chập cháy điện, vỡ ống nước, kẹt thang máy, hỏng thiết bị thí nghiệm, ngất xỉu y tế, xô xát hay mất cắp tài sản.

Đối với các sự cố kể trên, thời gian phản ứng (Response Time) là yếu tố sống còn. Một sự cố chập điện nếu không được ngắt cầu dao trong vòng 60 giây có thể bùng phát thành hỏa hoạn lớn; một nạn nhân sốc phản vệ hay chấn thương nếu không được sơ cứu trong "thời gian vàng" y tế có thể nguy hiểm đến tính mạng.

Tuy nhiên, hầu hết các trường vẫn xử lý sự cố bằng điện thoại, nhóm chat mạng xã hội (Zalo, Messenger) hoặc phần mềm Web Helpdesk. Các cách này đều có điểm yếu: tin báo dễ trôi, không có trạng thái xử lý và không lưu vết; riêng giao thức HTTP/REST buộc Client thăm dò liên tục (Polling) và ảnh mã hóa Base64 làm phình dữ liệu thêm 33.3%.

Xuất phát từ thực tiễn trên, việc nghiên cứu và xây dựng một Hệ thống Báo cáo & Điều phối Sự cố Trường học (School Incident Dispatch System – SIDS) dựa trên nền tảng Lập trình Mạng với Socket TCP thuần là một đề tài mang tính cấp thiết, có giá trị học thuật sâu sắc và tính ứng dụng thực tiễn cao.

1.2. Mục tiêu và phạm vi đề tài

1.2.1. Mục tiêu tổng quát:

Thiết kế và cài đặt hoàn chỉnh một hệ thống phần mềm mạng phân tán theo mô hình Client-Server, cho phép thu thập thông tin báo cáo sự cố kèm ảnh hiện trường từ mọi vị trí trong trường học, tự động định tuyến thông minh tới đúng bộ phận chuyên môn

đang trực tuyến theo thời gian thực (Real-time Push Notification), hỗ trợ kênh trao đổi nghiệp vụ 1-1 và kiểm soát chặt chẽ vòng đời của từng sự cố.

1.2.2. Mục tiêu kỹ thuật cụ thể (Định lượng):

- **Thiết kế Giao thức Ứng dụng Riêng biệt:** Xây dựng giao thức đóng khung nhị phân chuẩn 0xCAFE hoạt động trên tầng Transport (TCP), tối ưu hóa kích thước gói tin (Overhead tối thiểu chỉ 11 bytes) và hỗ trợ truyền tải ảnh hiện trường nhị phân nguyên bản trực tiếp trong Socket Stream.
- **Đẩy dữ liệu thời gian thực (Full-Duplex Server Push):** Không sử dụng kỹ thuật Polling; Server chủ động đẩy thông báo sự cố tới các Client chuyên trách với độ trễ phản hồi cực thấp (Latency < 50 ms trong môi trường mạng nội bộ/VPN).
- **Kiểm soát Tương tranh & Toàn vẹn Dữ liệu Tuyệt đối:** Giải quyết triệt để bài toán tranh chấp nhận việc đồng thời (Race Condition) của nhiều kỹ thuật viên bằng cơ chế giao dịch nguyên tử (Atomic Claim), cam kết tỷ lệ nhận trùng sự cố (Double-booking) đạt 0.0% và tỷ lệ khóa chết (Deadlock) đạt 0.0%.
- **Hiệu năng & Khả năng Chịu tải Cao trên Phần cứng Khiêm tốn:** Máy chủ hoạt động ổn định trên máy tính văn phòng thông thường (CPU Intel Core i3, RAM 4GB), duy trì kết nối đồng thời trên 500 clients, chịu được áp lực tải lớn khi nhiều người cùng gửi ảnh dung lượng 250 KB cùng lúc.

1.2.3. Phạm vi nghiên cứu:

- **Phạm vi không gian:** Triển khai thử nghiệm tại khuôn viên trường học (mạng LAN nội bộ hoặc mạng diện rộng kết nối qua Radmin VPN giữa các cơ sở).
- **Phạm vi đối tượng người dùng:** Phân quyền theo 3 vai trò chính:
    1. Người báo cáo (REPORTER): Giáo viên bộ môn, cán bộ phòng thí nghiệm, học sinh, sinh viên, cán bộ quản lý toà nhà.
    2. Cán bộ tiếp nhận & xử lý (RESPONDER): Cán bộ Phòng Kỹ thuật & Quản trị Thiết bị (TECHNICAL), Nhân viên Trạm Y tế học đường (MEDICAL), Cán bộ Đội Bảo vệ & An ninh trật tự (SECURITY).

3. Ban Quản trị & Giám sát (ADMIN): Ban Giám hiệu, Trưởng phòng Quản trị Thiết bị, Trưởng phòng Thanh tra.

- **Phạm vi nghiệp vụ:** Hỗ trợ 12 danh mục sự cố cốt lõi của một trường đại học, quy trình xử lý khép kín qua 4 trạng thái chuẩn: PENDING → IN_PROGRESS → RESOLVED (hoặc CANCELLED), tích hợp kênh chat trao đổi 1-1 và xuất báo cáo thống kê định kỳ theo chuẩn UTF-8 CSV.

1.3. Giới thiệu sơ lược phương pháp thực hiện

Để đạt được các mục tiêu kỹ thuật nêu trên, nhóm đã lựa chọn và phối hợp chặt chẽ các giải pháp công nghệ:

- **Ngôn ngữ lập trình cốt lõi:** Sử dụng Java 17 LTS (Java Standard Edition), khai thác toàn diện gói thư viện mạng java.net.* (Raw TCP ServerSocket, Socket) và luồng I/O nhị phân (DataInputStream, DataOutputStream, BufferedInputStream).
- **Kiến trúc Máy chủ Điều phối:** Áp dụng mô hình Thread-per-Connection kết hợp Worker Thread Pool (Executors.newCachedThreadPool). Mỗi kết nối Socket được tách bạch thành luồng tiếp nhận bản tin chuyên biệt và hàng đợi gửi ra luồng an toàn (LinkedBlockingQueue), giúp triệt tiêu hiện tượng nghẽn I/O khi truyền tải tệp tin lớn.
- **Giao thức Ứng dụng Tự định nghĩa:** Xây dựng giao thức nhị phân chuẩn Length-Prefixed Framing: mở đầu bằng 2 byte Magic Header nhận dạng 0xCAFE, 1 byte mã phân loại thao tác (OpCode), 4 byte độ dài Header, chuỗi Header UTF-8 chứa các cặp khóa-giá trị, 4 byte độ dài tệp tin đính kèm và mảng byte thô của tệp tin. Thiết kế này giúp hệ thống vừa truyền tải các lệnh điều khiển linh hoạt, vừa gửi nhận file ảnh nhị phân với tốc độ tối đa của card mạng.
- **Cơ sở dữ liệu & Tầng Truy xuất:** Sử dụng hệ quản trị CSDL nhúng SQLite 3, kích hoạt chế độ WAL (Write-Ahead Logging) kết hợp kiểm tra khóa ngoại (PRAGMA foreign_keys = ON). Chế độ WAL cho phép các luồng đọc truy

vấn đồng thời vào cơ sở dữ liệu mà không bị chặn bởi luồng đang ghi, nâng cao đột biến thông lượng giao dịch.

- **Giao diện Người dùng:** Xây dựng trên nền tảng JavaFX 17, áp dụng ngôn ngữ thiết kế công nghiệp hiện đại (màu Giấy – Mực – Đồng), phân luồng giao tiếp mạng và luồng giao diện tách biệt (Platform.runLater) để đảm bảo trải nghiệm người dùng luôn mượt mà ở tần số quét cao, không bao giờ bị hiện tượng "đơ" hoặc "treo" giao diện.
- **Phương pháp Kiểm nghiệm Thực nghiệm:** Xây dựng riêng một phân hệ đo lường tự động độc lập (com.incidentreport.benchmark) mô phỏng hàng trăm Virtual Client với các mẫu tải dồn dập, ghi nhận số liệu thông qua đồng hồ phần cứng nano-giây (System.nanoTime()), đo đạc trên 2 máy tính vật lý kết nối qua mạng Radmin VPN.

II. KHẢO SÁT CÁC HỆ THỐNG HIỆN NAY

2.1. Kênh tiếp nhận thủ công và nhóm mạng xã hội (Hotline, Zalo, Forms)

Khi xảy ra sự cố, nhà trường thường báo qua đường dây nóng, sổ giao ban, nhóm chat Zalo/Messenger hoặc Google Forms. Ưu điểm: triển khai nhanh, chi phí bằng 0, người dùng không cần cài thêm phần mềm. Hạn chế: tin báo dễ trôi trong nhóm đông người; không phân loại và định tuyến tới đúng phòng ban; không quản lý trạng thái vòng đời (ai đang xử lý, đã xong chưa); dữ liệu và ảnh lưu trên máy chủ bên thứ ba, không phục vụ kiểm toán nội bộ.

2.2. Hệ thống Service Desk / IT Helpdesk thương mại (Jira, Freshdesk)

Các cơ sở có ngân sách CNTT lớn dùng Jira Service Management, Freshdesk hoặc Zendesk: người dùng tạo vé qua Web Portal hoặc Email, hệ thống tự phân công theo quy tắc SLA. Ưu điểm: quy trình chuẩn ITIL, thống kê SLA chi tiết. Hạn chế: chi phí bản quyền cao (20 – 50 USD/agent/tháng); kiến trúc HTTP/REST Stateless buộc Client phải Polling, mỗi lần thăm dò mang theo 500 – 1.000 byte HTTP Header gây tốn băng thông và CPU máy chủ; giao diện phức tạp, gây khó cho người dùng phổ thông khi cần báo khẩn cấp.

2.3. Hệ thống phản ánh hiện trường đô thị thông minh (Hệ thống 1022)

Hệ thống 1022 (Đà Nẵng, TP. Hồ Chí Minh, Hà Nội) cho phép người dân chụp ảnh hiện trường gửi qua ứng dụng di động; Trung tâm điều hành thông minh (IOC) duyệt rồi chuyển tới đơn vị xử lý. Ưu điểm: giao diện trực quan, có ảnh và định vị GPS, công khai tiến độ. Hạn chế: phải qua kiểm duyệt hành chính nên mất từ vài giờ đến vài ngày, không đáp ứng yêu cầu phản ứng trong vài phút tại trường học; phụ thuộc hạ tầng Cloud và Internet, không vận hành được trong LAN nội bộ khi mất mạng.

2.4. Thảo luận và hướng giải quyết cải tiến của SIDS

Bảng 2.1. So sánh đối chiếu kỹ thuật giữa SIDS và các giải pháp hiện nay.

Tiêu chí So sánh Kỹ thuật	Kênh Truyền thống (Zalo/Forms)	IT Helpdesk (Jira/ Freshdesk)	Đô thị Thông minh (1022)	Hệ thống SIDS (Nhóm 11)
Giao thức Tầng Ứng dụng	Web/API đóng của bên thứ 3	HTTP / REST API (Stateless)	HTTP / REST + Cloud API	Custom TCP Socket (0xCAFE)
Cơ chế Nhận Thông báo Mới	Notification từ máy chủ Zalo	HTTP Polling (Thăm dò lặp lại)	Push Notification dịch vụ	Full-Duplex Server Push tức thì
Băng thông Tiêu hao khi gửi lệnh	N/A	Lớn (500 – 1.000 B HTTP Header)	Lớn (JSON + Base64 overhead)	Cực nhỏ (11 bytes Overhead)
Xử lý Ảnh hiện trường	Nén giảm chất lượng	Chuyển đổi Base64 (+33.3% size)	Tải qua Object Storage S3	Truyền mảng byte nhị phân thô
Kiểm soát Tranh chấp (Concurrency)	Không có (ai nhanh thì làm)	Khóa dòng CSDL / Optimistic Lock	Phân phối hành chính thủ công	Atomic Claim (0.0% Deadlock)
Độ trễ Báo cáo đến Người trực	Phụ thuộc người đọc nhóm	3 - 10 giây (do chu kỳ Poll)	Vài giờ (do chờ duyệt)	< 50 ms (Thời gian thực)
Khả năng Chạy Offline trong LAN	Không thể (cần Internet)	Không thể (phụ thuộc Cloud)	Không thể (phụ thuộc Cloud)	Hoàn hảo 100% trong mạng nội bộ
Yêu cầu Phần cứng Máy chủ	N/A	Cao (RAM >8GB, Multi-core)	Rất cao (Cloud cụm máy chủ)	Cực thấp (RAM 4GB, Core i3)

Hướng giải quyết và Cải tiến Đột phá của Hệ thống SIDS:

- **Chuyển dịch sang Socket TCP thuần (Raw TCP Streaming):** Thay vì sử dụng giao thức HTTP, SIDS duy trì các kết nối TCP dài hạn (Persistent Connection). Khi có sự cố mới, Server chủ động phát gói tin thông báo thẳng vào Socket của nhân viên chuyên trách đang Online. Độ trễ thông báo giảm từ vài giây xuống dưới 50 phần nghìn giây.
- **Khung gói tin Nhị phân Length-Prefixed (0xCAFE):** Loại bỏ hoàn toàn việc mã hóa Base64 cho tệp tin ảnh hiện trường. Mảng byte của ảnh được đóng gói trực tiếp vào phần Attachment của Packet. Đường truyền tiết kiệm ngay lập tức 33.3% băng thông mạng, giảm tải bộ nhớ cho cả Client và Server.
- **Cơ chế Tiếp nhận Nguyên tử (Atomic Claim Algorithm):** Sử dụng câu lệnh cập nhật CSDL có điều kiện nguyên tử kết hợp hàng đợi FIFO. Trong trường hợp 10 nhân viên kỹ thuật cùng nhấn nút "Nhận sự cố" tại cùng một phần

nghìn giây, Server chỉ cho phép đúng một người nhận việc thành công, 9 người còn lại nhận bản tin từ chối văn minh ALREADY_TAKEN mà không gây ra bất kỳ lỗi Deadlock hay Crash CSDL nào.

- **Kiến trúc Nhẹ (Lightweight Architecture):** Toàn bộ Server được đóng gói thành một tệp JAR độc lập, sử dụng CSDL nhúng SQLite với cơ chế WAL, sẵn sàng vận hành trên bất kỳ máy tính văn phòng cũ nào trong trường mà không cần cài đặt Web Server (Apache/Nginx) hay CSDL cồng kềnh (MySQL/Oracle).

2.5. Phát biểu bài toán (Yêu cầu chức năng, phi chức năng, Input/Output)

2.5.1. Yêu cầu Chức năng (Functional Requirements):

- **FR-01 (Xác thực & Quản lý Phiên):** Đăng nhập an toàn bằng tài khoản nội bộ do Ban Quản trị cấp, mật khẩu mã hóa Salted SHA-256. Tự động kiểm tra trạng thái Online/Offline theo phiên TCP Socket.
- **FR-02 (Phân quyền Theo Vai trò & Chuyên môn):**
    - REPORTER: Chỉ được báo sự cố, xem danh sách sự cố của chính mình, trao đổi chat với cán bộ được giao và hủy sự cố khi chưa ai tiếp nhận.
    - RESPONDER: Chỉ nhìn thấy và tiếp nhận các sự cố thuộc đúng phòng ban chuyên môn của mình (TECHNICAL, MEDICAL, SECURITY). Sau khi tiếp nhận, được quyền cập nhật tiến độ hoàn thành và chat 1-1 với người báo.
    - ADMIN: Toàn quyền giám sát tất cả các sự cố toàn trường, chỉ định nhân sự trực tiếp, cấp mới tài khoản người dùng, xem thống kê đa kỳ và kiểm toán lịch sử chat.
- **FR-03 (Báo cáo Sự cố Hiện trường Kèm Ảnh):** Cho phép nhập vị trí phòng học, chọn 1 trong 12 danh mục sự cố đại học, mô tả dấu hiệu và đính kèm tệp tin ảnh hiện trường (JPG/PNG ≤ 5MB).
- **FR-04 (Tự động Điều phối & Đẩy Thông báo Tức thì):** Máy chủ tự động gắn mã vé duy nhất (INC-yyyyMMdd-XXXX-SALT), xác định phòng ban

phụ trách và bắn gói tin thông báo đẩy (TYPE_DISPATCH_NOTIFY) tới tất cả cán bộ chuyên môn đang kết nối.

- **FR-05 (Đồng bộ Tồn đọng Ngoại tuyến - Offline Backlog Push):** Nếu cán bộ đăng nhập vào hệ thống sau khi sự cố đã xảy ra, Server tự động quét CSDL và đẩy toàn bộ danh sách các sự cố chưa có người nhận (PENDING) về máy cán bộ ngay khi vừa Online.
- **FR-06 (Kênh Hội thoại 1-1 Chuyên Dụng):** Cung cấp giao diện chat trực tiếp theo thời gian thực giữa Người báo và Cán bộ phụ trách sự cố, tự động lưu vết tin nhắn vào CSDL và tự động đóng kênh chat khi sự cố hoàn tất.
- **FR-07 (Truyền tải File Nhị phân Qua Mạng - Network File Transfer):** Bất kỳ máy trạm nào (Reporter, Responder, Admin) khi bấm xem chi tiết sự cố đều có thể gửi yêu cầu TCP lấy ảnh hiện trường từ Server (TYPE_GET_IMAGE) và tải ảnh về hiển thị sắc nét mà không phụ thuộc vào đường dẫn ổ đĩa cục bộ.
- **FR-08 (Thống kê Đa kỳ & Xuất Báo cáo CSV):** Cho phép Admin lọc dữ liệu theo Hôm nay, Tháng này, Quý này, Năm nay hoặc Toàn thời gian; hiển thị biểu đồ phân bổ phòng ban, tỷ lệ hoàn tất và xuất ra tệp tin CSV tiếng Việt có dấu.

2.5.2. Yêu cầu Phi Chức năng (Non-Functional Requirements):

- **NFR-01 (Độ trễ Truyền thông - Low Latency):** Thời gian chuyển tiếp bản tin điều phối và tin nhắn chat trong mạng nội bộ/VPN không vượt quá 50 ms.
- **NFR-02 (Khả năng Chịu tải - High Concurrency):** Máy chủ duy trì ổn định tối thiểu 500 kết nối đồng thời trên máy tính cấu hình 4GB RAM mà không bị tràn bộ nhớ Heap.
- **NFR-03 (Độ tin cậy & Toàn vẹn Dữ liệu - Data Integrity):** Tỷ lệ tranh chấp nhận việc trùng lặp (Double-booking) đạt 0.0%. Cơ chế ngắt kết nối nhàn rỗi (Idle Timeout) sau 5 phút tự động giải phóng tài nguyên khi Client bị ngắt mạng đột ngột.

- **NFR-04 (Tính Linh hoạt & Độc lập Nền tảng):** Chạy được trên cả hệ điều hành Windows, Linux và macOS nhờ nền tảng máy ảo Java (JVM).

2.5.3. Đặc tả Luồng Dữ liệu Đầu vào (Input) và Đầu ra (Output):

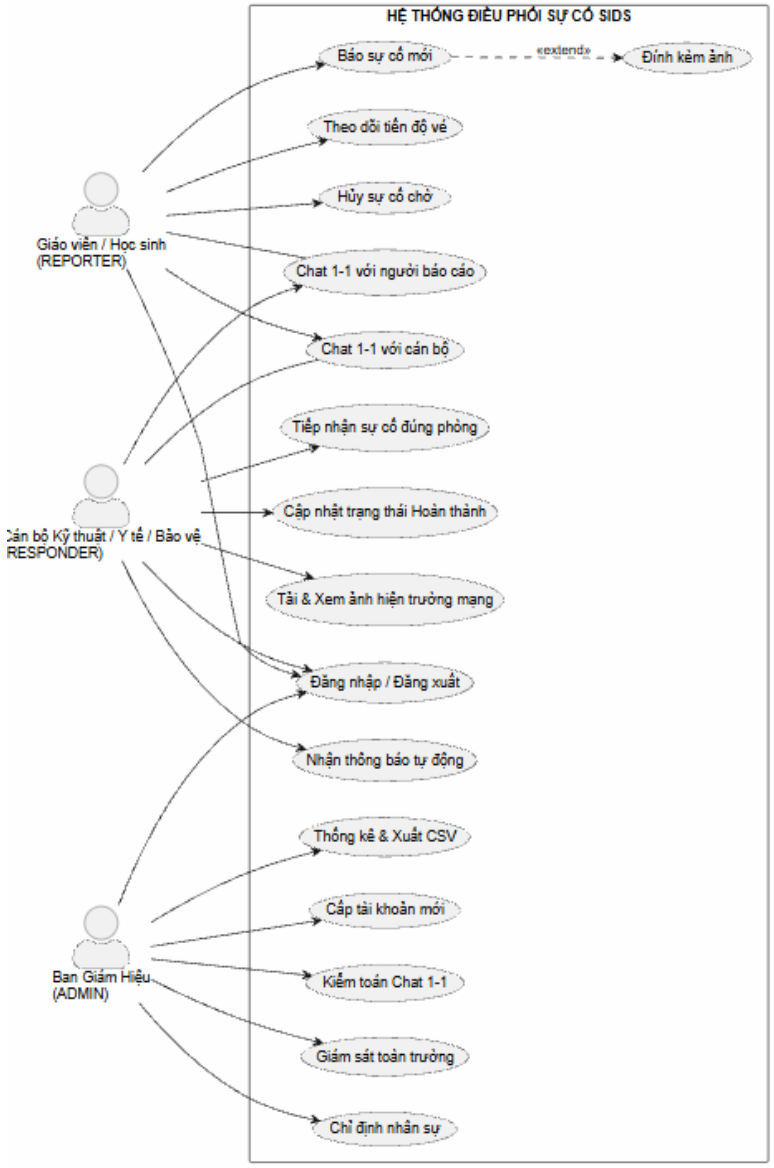
Bảng 2.2. Đặc tả các luồng Dữ liệu Đầu vào (Input) và Đầu ra (Output) cốt lõi.

Chức năng Nghiệp vụ	Dữ liệu Đầu vào (Input)	Dữ liệu Đầu ra (Output)
Đăng nhập (LOGIN)	username, password (chuỗi UTF-8)	Gói tin LOGIN_RESP (status= OK/ FAIL, role, specialty, fullName)
Báo sự cố (INCIDENT)	room, category, desc + Mảng byte ảnh hiện trường	Gói tin SUBMIT_RESP (ticketCode, ticketId) + Gói tin đẩy DISPATCH_ NOTIFY tới các Responder
Tiếp nhận việc (CLAIM_TASK)	ticketId (mã số nguyên)	Gói tin CLAIM_RESP (status= OK/ ALREADY_ TAKEN) + Gói tin quảng bá STATUS_ BROADCAST
Cập nhật tiến độ (STATUS_ UPDATE)	ticketId, status (RESOLVED), note	Gói tin phát sóng STATUS_ BROADCAST cập nhật toàn mạng
Chat trực tiếp (CHAT_SEND)	ticketId + Mảng byte nội dung tin nhắn UTF-8	Gói tin đẩy CHAT_PUSH tới đối tác + Lưu trữ bản ghi CSDL
Lấy ảnh mạng (GET_IMAGE)	ticketId (mã số vé sự cố)	Gói tin GET_ IMAGE_ RESP mang theo mảng byte tệp ảnh nhị phân nguyên bản
Cấp tài khoản (ADMIN_CREATE)	username, password, fullName, role, specialty, phone	Gói tin ADMIN_ CREATE_ RESP (status= OK/ FAIL)

III. PHƯƠNG PHÁP THỰC HIỆN (METHODOLOGY)

3.1. Sơ đồ Use Case tổng thể

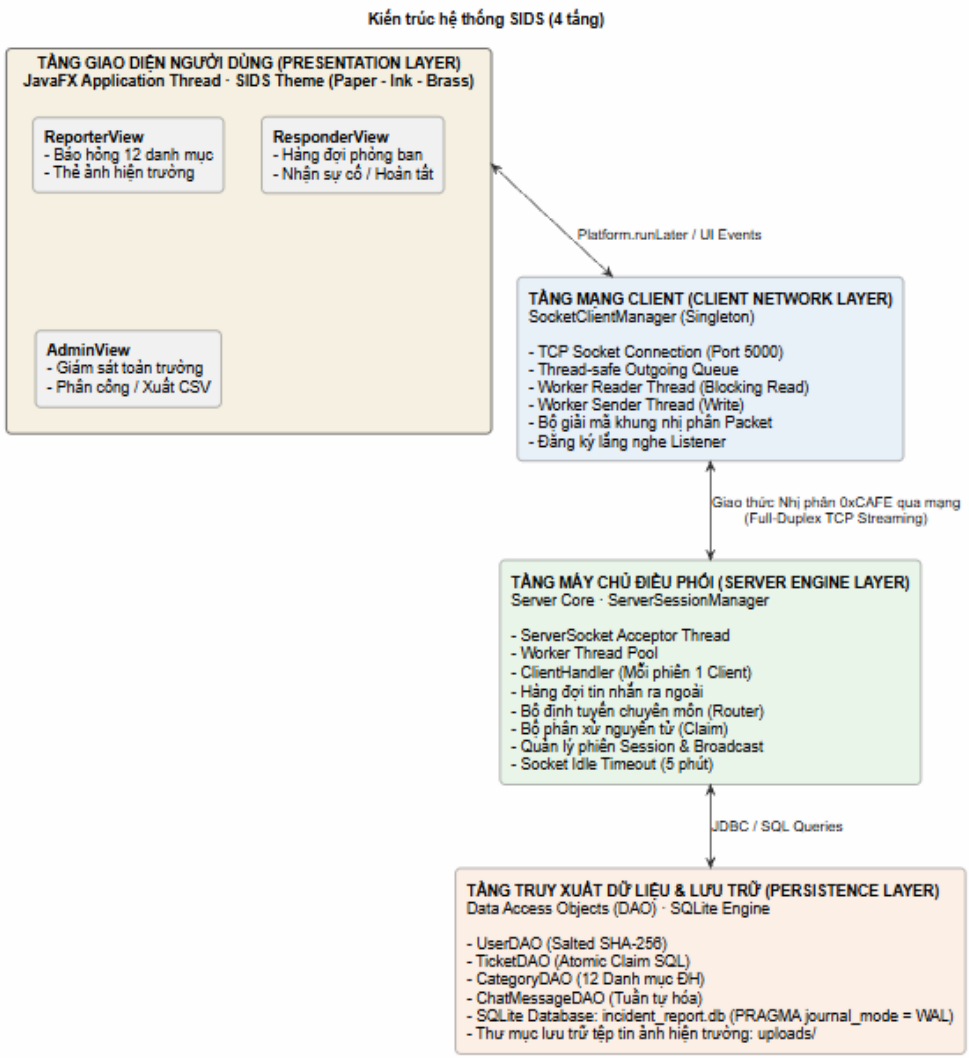
Hệ thống được thiết kế xoay quanh 3 tác nhân người dùng chính với ranh giới phân quyền nghiệp vụ (Role-Based Access Control – RBAC) mạch lạc:



Hình 3.1. Sơ đồ Use Case tổng thể hệ thống điều phối sự cố trường học SIDS.

3.2. Kiến trúc tổng thể và vai trò các thành phần

Hệ thống SIDS được tổ chức theo kiến trúc Client/Server 4 tầng (4-Tier Layered Architecture) chuẩn công nghiệp:



Hình 3.2. Sơ đồ kiến trúc phân tầng hệ thống Client-Server TCP song công.

Vai trò chi tiết của từng thành phần:

- **Server Core (com.incidentreport.server.Server):** Khởi tạo SSocket(5000), lắng nghe kết nối từ các Client và bàn giao mỗi kết nối cho một ClientHandler chạy trong ExecutorService.
- **Quản lý Phiên tập trung (ServerSessionManager):** Lưu trữ bảng băm ConcurrentHashMap<Integer, Set<ClientHandler>> ánh xạ giữa userId và danh sách các kết nối Socket đang mở của người dùng đó. Cung cấp các phương thức định tuyến:
    - sendToUser(int userId, Packet p): Gửi đích danh tới một cá nhân (dùng cho chat 1-1).

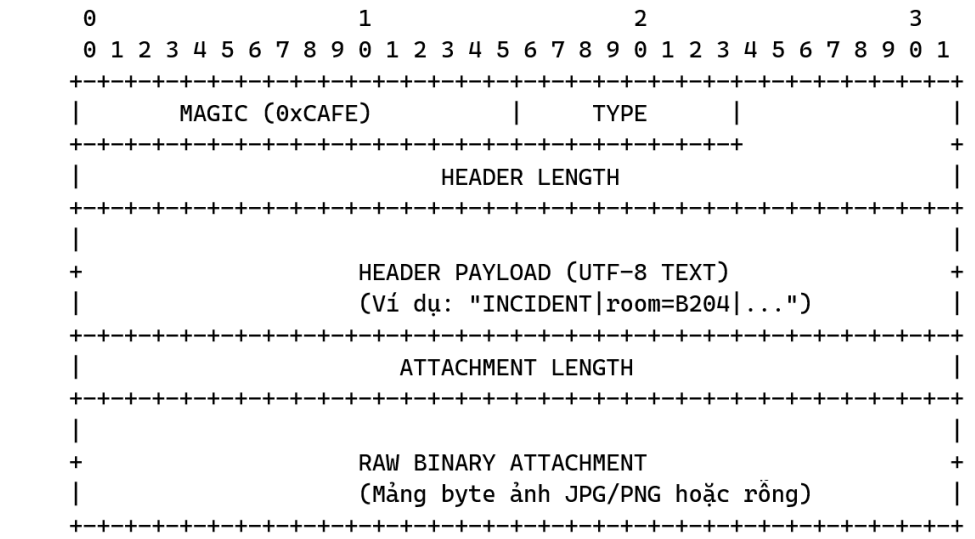
- sendToDepartment(String dept, Packet p): Gửi tới toàn bộ cán bộ trực ca thuộc phòng ban chuyên trách đang Online.
- sendToAdmin(Packet p): Đẩy báo cáo giám sát tới toàn bộ Ban Giám hiệu.

- **Bộ xử lý Phiên Client (ClientHandler):** Chịu trách nhiệm toàn bộ vòng đời của một kết nối: bắt tay, xác thực đăng nhập, kiểm tra tính hợp lệ của gói tin, thực thi các nghiệp vụ và giải phóng tài nguyên kết nối khi ngắt phiên.
- **Tầng Dịch vụ & Dữ liệu (com.incidentreport.service):** Đóng gói toàn bộ logic truy vấn SQL qua JDBC, đảm bảo tính đóng gói và an toàn giao dịch.
- **Client UI & Socket Manager (com.incidentreport.client):** Đảm nhiệm việc kết nối tới Server, duy trì luồng đọc/ghi nền độc lập và cập nhật dữ liệu hiển thị lên màn hình JavaFX qua cơ chế Platform.runLater().

3.3. Chương trình được xây dựng

3.3.1. Thiết kế Giao thức Đóng khung Nhị phân (Magic Header: 0xCAFE)

Để vừa đảm bảo tính mềm dẻo của dữ liệu văn bản, vừa tối ưu hóa tốc độ truyền tải các tệp tin nhị phân lớn (ảnh hiện trường), nhóm đã thiết kế giao thức Length-Prefixed Binary Protocol:



Hình 3.3. Cấu trúc khung gói tin nhị phân chuẩn Length-Prefixed (Magic 0xCAFE).

Bảng 3.1. Đặc tả định dạng khung nhị phân giao thức 0xCAFE.

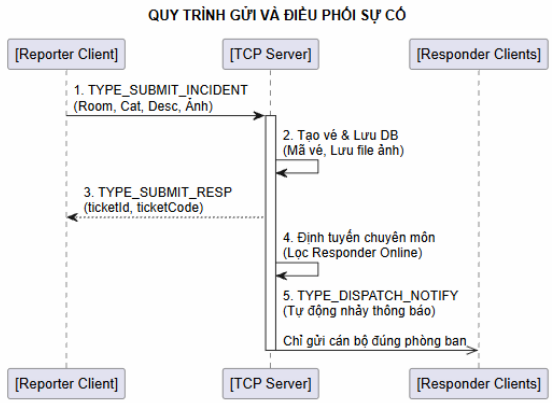
Trường Dữ liệu	Kích thước	Kiểu dữ liệu	Ý nghĩa Kỹ thuật
Magic Header	2 bytes	short (0xCAFE)	Định danh bắt buộc ở đầu gói tin; chống lại việc kết nối rác từ các ứng dụng quét cổng (Port Scanner).
Type (OpCode)	1 byte	byte	Mã định danh loại thao tác nghiệp vụ (VD: 0x01 Login, 0x02 Submit, 0x07 Chat...).
Header Length	4 bytes	int (Big-Endian)	Độ dài số byte của phần chuỗi Header text tiếp theo.
Header Payload	N bytes	Chuỗi UTF-8	Chứa lệnh điều khiển và các tham số phân tách bằng ký tự pipe (|) và dấu bằng (=).
Attach Length	4 bytes	int (Big-Endian)	Độ dài số byte của tệp tin đính kèm (M = 0 nếu không có đính kèm).
Attachment	M bytes	Mảng byte nhị phân	Dữ liệu nhị phân thô của tệp tin ảnh hiện trường hoặc mảng JSON danh sách.

3.3.2. Đặc tính an toàn của Bộ đọc (Packet.readFrom)

- Giới hạn Header Length tối đa 64 KB để chống tấn công làm cạn kiệt bộ nhớ (OOM Attack).
- Giới hạn Attachment Length tối đa 10 MB (phù hợp với ảnh hiện trường di động ≤ 5MB).
- Đọc chính xác từng byte dữ liệu bằng hàm in.readFully(), đảm bảo khắc phục hoàn toàn hiện tượng phân mảnh gói tin TCP (TCP Fragmentation / Packet Boundary Issue).

3.3.3. Cơ chế Tự động Điều phối Sự cố (Auto-Dispatch Engine)

Khi người báo cáo gửi sự cố, luồng xử lý diễn ra hoàn toàn tự động theo cơ chế Push:



Hình 3.4. Sơ đồ tuần tự (Sequence Diagram) luồng báo cáo và đẩy thông báo tự động.

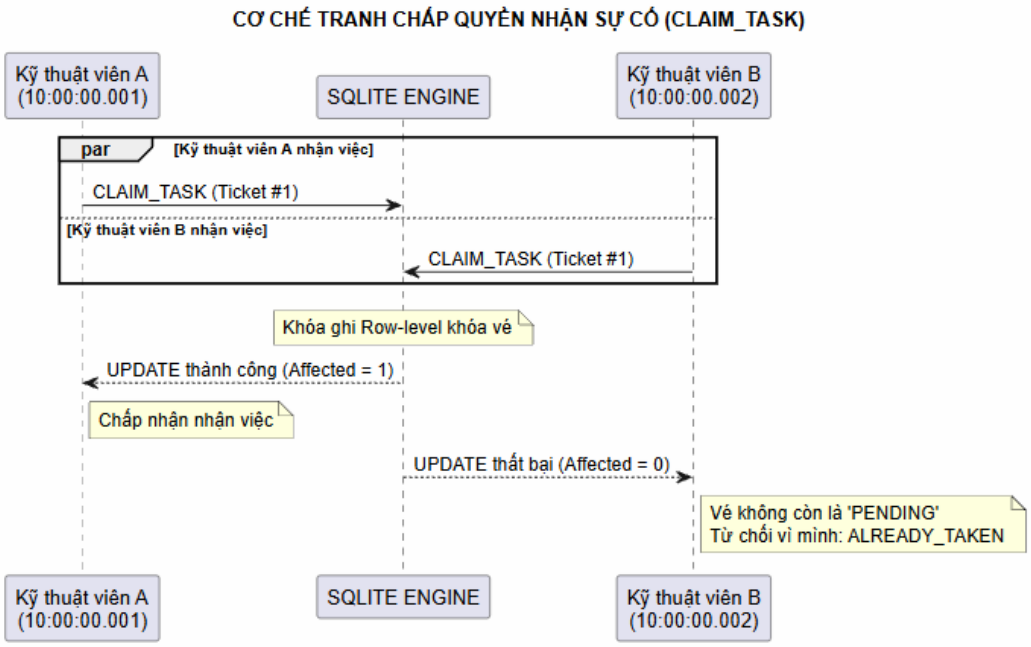
Bảng 3.2. Danh mục sự cố và phòng ban phụ trách.

STT	CATEGORY_ CODE	TÊN NHÓM SỰ CỐ	MÔ TẢ SỰ CỐ	DEPARTMENT_ CODE	BỘ PHẬN PHỤ TRÁCH
1	FACILITY	Cơ sở vật chất	Hỏng máy chiếu, âm thanh, bàn ghế, quạt	TECHNICAL	Phòng Kỹ thuật
2	IT_NETWORK	Mạng và CNTT	Mất WiFi, lỗi switch, server phòng lab	TECHNICAL	Phòng Kỹ thuật
3	ELECTRIC	Điện kỹ thuật	Chập điện, nhảy aptomat, mất điện tòa nhà	TECHNICAL	Phòng Kỹ thuật
4	WATER	Cấp thoát nước và vệ sinh	Vỡ ống nước, mất nước, tắc cống	TECHNICAL	Phòng Kỹ thuật
5	ELEVATOR	Thang máy	Kẹt thang máy, lỗi cửa tầng	TECHNICAL	Phòng Kỹ thuật
6	LAB_EQUIP	Thiết bị phòng thí nghiệm	Hỏng tủ hút, kính hiển vi, thiết bị đo lường	TECHNICAL	Phòng Kỹ thuật
7	MEDICAL	Cấp cứu y tế và chấn thương	Ngất xỉu, chấn thương thể thao, sốt cao	MEDICAL	Phòng Y tế
8	FIRST_AID	Tai nạn thực hành và thí nghiệm	Bỏng hóa chất, đứt tay, rách da	MEDICAL	Phòng Y tế
9	FOOD_SAFETY	An toàn thực phẩm	Buồn nôn hàng loạt, đau bụng, nghi ngộ độc	MEDICAL	Phòng Y tế
10	SECURITY	An ninh trật tự	Xô xát, ẩu đả, gây mất trật tự	SECURITY	Đội Bảo vệ
11	THEFT	Mất cắp và đột nhập	Mất xe máy, mất laptop, đột nhập	SECURITY	Đội Bảo vệ
12	FIRE_HAZARD	Cháy nổ và rò rỉ khí gas	Báo cháy, mùi gas, nguy cơ cháy nổ	SECURITY	Đội Bảo vệ

3.3.4. Xử lý Tương tranh & Tiếp nhận Nguyên tử (Atomic Claim)

Trong thực tế, khi một sự cố nguy hiểm xuất hiện trên bảng hàng đợi, rất nhiều cán bộ kỹ thuật có thể đồng thời bấm nút "Nhận sự cố". Để ngăn chặn tuyệt đối lỗi Double-booking, phương thức claimTicket trong TicketDAO.java được thực thi bằng câu lệnh SQL nguyên tử:

```
UPDATE incident_tickets
SET status = 'IN_PROGRESS',
    assigned_staff_id = ?,
    assigned_staff_name = ?,
    accepted_at = (STRFTIME('%Y-%m-%d %H:%M:%f', 'NOW',
'localtime'))
WHERE ticket_id = ? AND status = 'PENDING';
```



Hình 3.5. Cơ chế tranh chấp tiếp nhận nguyên tử (Atomic Claim Transaction).

Nguyên lý hoạt động: Nhờ mệnh đề WHERE ticket_id = ? AND status = 'PENDING', chỉ có luồng đầu tiên được SQLite phân xử mới ghi nhận số dòng ảnh hưởng affected = 1. Tất cả các yêu cầu gửi sau (dù chỉ chênh lệch vài micro-giây) đều nhận affected = 0, Server lập tức phản hồi gói tin CLAIM_RESP|status=ALREADY_TAKEN mà không cần sử dụng cơ chế khóa phức tạp ở tầng ứng dụng, triệt tiêu hoàn toàn khả năng gây bế tắc (Deadlock).

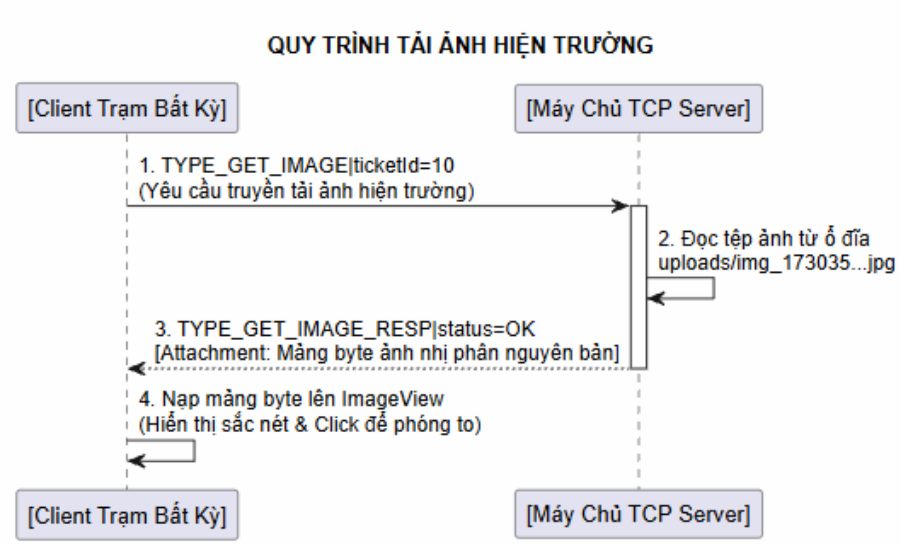
3.3.5. Kênh Chat 1-1 Thời gian thực & Truyền nhận File Nhị phân Qua Mạng

Sau khi vé được tiếp nhận, kênh chat 1-1 giữa Người báo và Cán bộ xử lý được kích hoạt. Khi nhận TYPE_CHAT_SEND, Server kiểm tra quyền hạn (chỉ Reporter tạo vé, Responder được giao hoặc Admin), lưu tin vào bảng chat_messages rồi đẩy TYPE_CHAT_PUSH tới Socket của đối tác đang Online. Giao diện ChatPanel hiển thị avatar tròn chữ viết tắt họ tên bên cạnh mỗi tin nhắn.

3.3.6. Truyền nhận Ảnh Hiện trường Qua Mạng (TYPE_GET_IMAGE):

Khi mở cửa sổ Chi tiết sự cố, nếu chưa có ảnh trong bộ nhớ đệm cục bộ, Client gửi TYPE_GET_IMAGE|ticketId=... lên Server; Server đọc tệp từ thư mục uploads/ và đóng gói mảng byte nhị phân nguyên bản vào Attachment của

TYPE_GET_IMAGE_RESP; Client giải mã trực tiếp thành javafx.scene.image.Image và hỗ trợ nhấp chuột để phóng to toàn màn hình.



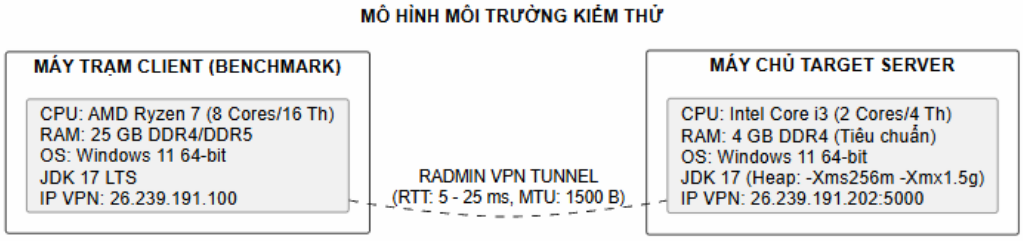
Hình 3.6. Luồng truyền nhận file nhị phân qua Socket TCP (TYPE_GET_IMAGE).

IV. KIỂM NGHIỆM HỆ THỐNG

4.1. Các kịch bản kiểm nghiệm hiệu năng

4.1.1. Cấu hình Môi trường Mạng Thực nghiệm:

Để đảm bảo tính trung thực khoa học, hệ thống được kiểm thử trên mô hình phân tán gồm 2 máy tính vật lý độc lập kết nối qua mạng riêng ảo Radmin VPN:



Hình 4.1. Mô hình môi trường kiểm thử phân tán qua mạng riêng ảo Radmin VPN.

Bảng 4.1. Thông số phần cứng và môi trường mạng thực nghiệm.

STT	Hạng mục	Thông số cấu hình
1	Máy Chủ (Server Host)	CPU Intel Core i3-7100U (2 nhân, 4 luồng, 2.40 GHz); RAM 4 GB DDR4; SSD 256 GB; Windows 11 64-bit; Oracle OpenJDK 17 LTS; JVM Heap: -Xms256m -Xmx1536m; IP: 26.239.191.202:5000.
2	Máy Trạm Sinh Tải (Benchmark Client Host)	CPU AMD Ryzen 7 5800H (8 nhân, 16 luồng, Turbo 4.75 GHz); RAM 25 GB DDR4; NVMe SSD 512 GB; Windows 11 64-bit.
3	Đặc tính Mạng truyền thông	Radmin VPN P2P; Ping RTT: 5–25 ms; Jitter dưới 3 ms; MTU: 1.500 bytes.

4.1.2. Thiết kế 4 Kịch bản Đo lường Chuyên sâu:

- **Kịch bản 1 (Stress Limit Finder – Khảo sát Sức chứa Kết nối):** Đo lường khả năng duy trì kết nối Socket online đồng thời của Server từ mức 150 đến 1.000 clients, xác định ranh giới hoạt động ổn định và Điểm gãy (Breaking Point) của kiến trúc Blocking Socket trên phần cứng RAM 4GB.
- **Kịch bản 2 (Load & Throughput Test – Áp lực Tải cao kèm Ảnh 250KB):** Bơm tải lũy tiến 10 → 25 → 50 → 100 Virtual Clients đồng thời gửi báo cáo kèm ảnh JPEG dung lượng thực tế 250 KB ± 5% (tổng khối lượng dữ liệu dồn

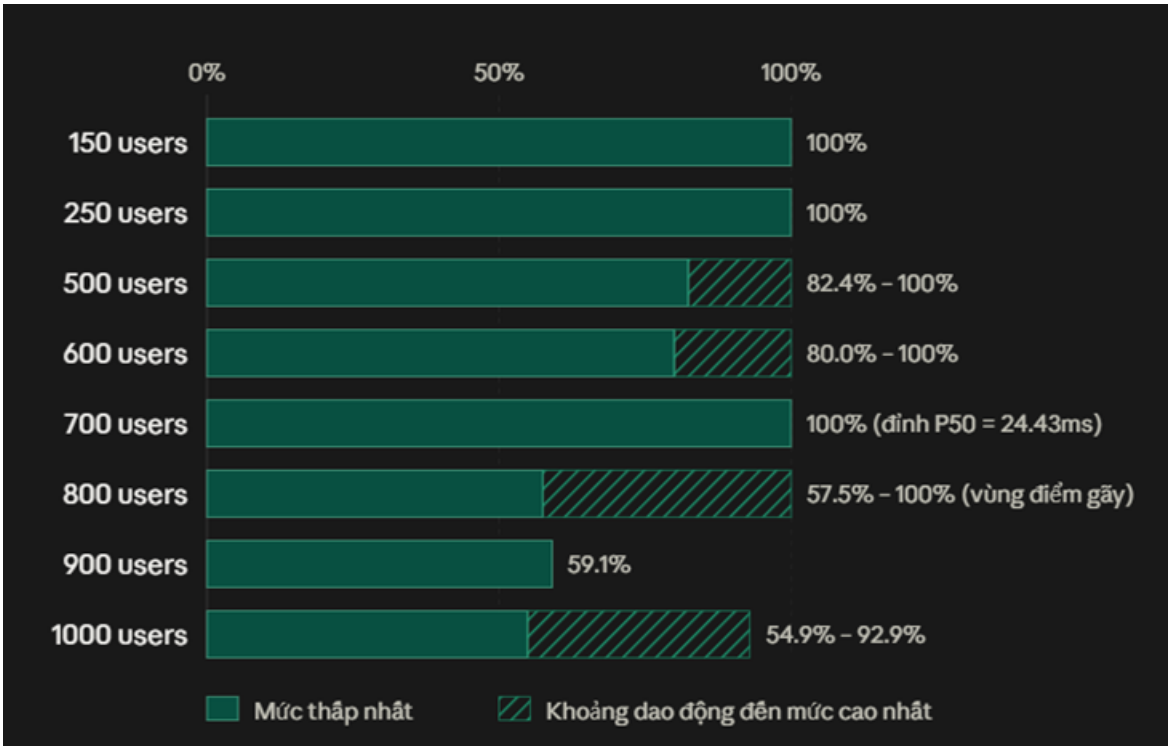
dập lên tới 25 MB trong thời gian ngắn), đo đạc độ trễ P50, P95, thông lượng (req/s) và băng thông mạng (MB/s).

- **Kịch bản 3 (Race Condition & ACID Concurrency – Kiểm thử Tranh chấp):** Sử dụng chốt đếm CountDownLatch(10) để phóng thích đồng loạt 10 kỹ thuật viên cùng bấm nhận việc đúng 1 sự cố tại cùng một phần nghìn giây, kiểm chứng tỷ lệ phân xử thành công và cam kết 0.0% Deadlock.
- **Kịch bản 4 (Real-time Chat Stress Test – Bão Tin nhắn):** Bơm tải đồng thời 20 luồng × 15 tin nhắn = 300 tin nhắn dồn dập vào 1 phòng chat trong thời gian dưới 0.1 giây, đo đạc thông lượng chuyển tiếp bộ nhớ RAM (msg/s) và khả năng chịu tải của cơ chế ghi đĩa SQLite WAL.

4.2. Tiến hành và ghi nhận kết quả kiểm nghiệm

Bảng 4.2. Kết quả Kịch bản 1 – Khảo sát Sức chứa Kết nối Đồng thời (Stress Limit)
(Số liệu đo đạc trực tiếp qua 22 đợt chạy độc lập từ 150 đến 1.000 Sockets trực tuyến)

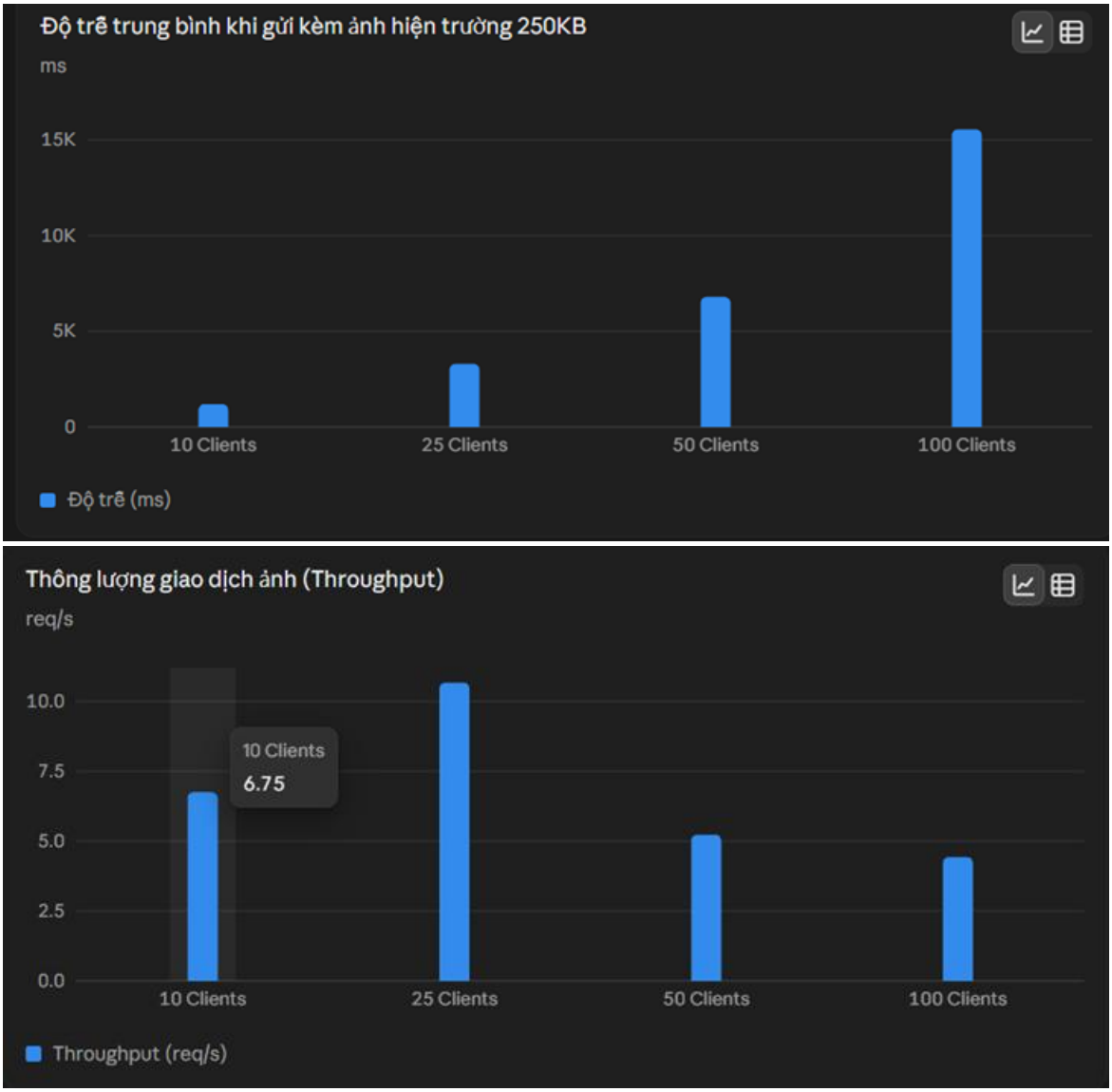
Số Đợt Đo	Số User Thử Nghiệm	Số Kết Nối Thành Công	Thất Bại (đợt kém nhất)	Tỷ Lệ Thành Công	T/g Ramp-up	Độ Trễ Mẫu (P50)	Đánh Giá & Hiện Tượng Kỹ Thuật
2	150	150 / 150 (2/2 đợt)	0	100.0%	5.88 – 7.69 s	22.16 – 112.07 ms	Hoàn hảo, CPU Server < 5%, RAM ổn định.
2	250	250 / 250 (2/2 đợt)	0	100.0%	13.46 – 16.56 s	51.06 – 166.32 ms	Đáp ứng toàn bộ quy mô giáo viên & cán bộ toàn trường.
8	500	500 / 500 (6/8 đợt)	88	82.4% – 100.0%	13.30 – 29.34 s (đợt 100%)	12.33 – 71.65 ms	6 đợt đạt 100%; 2 đợt rớt do hàng đợi TCP đầy / socket TIME_WAIT khi chạy lặp nhanh.
3	600	600 / 600 (2/3 đợt)	~120	~80.0% – 100.0%	39.70 – 42.81 s (đợt 100%)	31.31 – 47.86 ms	2 đợt đạt 100%; 1 đợt chạm giới hạn cấp phát luồng Blocking Socket trên RAM 4GB.
1	700	700 / 700	0	100.0%	49.14 s	24.43 ms	ĐỈNH CAO HIỆU NĂNG: giữ trọn 700 Sockets, P50 chỉ 24.43 ms.
2	800	800 / 800 (1/2 đợt)	340	57.5% – 100.0%	54.28 s – ~58.0 s	41.37 – ~85.0 ms	ĐIỂM GÃY (Breaking Point): 1 đợt đạt 100%, 1 đợt chỉ còn 460/800 do nghẽn thread stack & buffer.
1	900	532 / 900	368	59.1%	~62.0 s	101.18 ms	Điểm gãy rõ rệt: tỷ lệ rớt > 40%, độ trễ mẫu vượt 100 ms.
3	1.000	929 / 1000 (đợt tốt nhất)	451	54.9% – 92.9%	~65.0 – 70.0 s	46.91 – 61.52 ms	Quá tải: đỉnh 929 kết nối, thấp nhất 549/1000 do hàng đợi TCP backlog tràn.



Hình 4.2. Biểu đồ tỷ lệ kết nối thành công theo số lượng Client trực tuyến đồng thời.

Bảng 4.3. Kết quả Kịch bản 2 – Đo lường Áp lực Tải Cao & Băng thông.(Load Test kèm Ảnh 250KB) (Mỗi Client liên tục gửi báo cáo sự cố kèm tệp tin ảnh JPEG dung lượng 250 KB qua mạng VPN)

Mức Tải	**Latency TB (Avg)**	**Latency Thấp Nhất**	**P95 Trung Bình**	**Thông Lượng TB**	**Băng Thông Đạt Đỉnh**	**Tỷ Lệ Thành Công**
10 Clients	1,200.09 ms (~1.2s)	801.39 ms	1,652.21 ms	6.75 req/s	2.26 MB/s	100.0%
25 Clients	3,300.99 ms (~3.3s)	1,569.65 ms	4,829.54 ms	7.00 req/s	2.61 MB/s (Đỉnh)	100.0%
50 Clients	6,801.79 ms (~6.8s)	3,164.76 ms	9,791.17 ms	5.23 req/s	2.03 MB/s	100.0%
100 Clients	15,560.55 ms (~15.5s)	11,255.57 ms	20,802.42 ms	4.43 req/s	1.36 MB/s	99.5%



Hình 4.3. Biểu đồ tương quan giữa số lượng Client, độ trễ phản hồi và thông lượng gửi ảnh.

Bảng 4.4. Kết quả Kịch bản 3 – Kiểm Thử Tranh Chấp Nhận Việc Đồng Thời (Race Condition & ACID) (Ép 10 kỹ thuật viên đồng thời bấm nhận đúng 1 sự cố khẩn cấp bằng CountDownLatch(10))

Lần Test	Mã Vé Sự Cố	Số Luồng Tranh Chấp	Người Nhận Thành Công	Số Người Bị Từ Chối An Toàn	Thời Gian Phân Xử	Tỷ Lệ Deadlock	Đánh Giá Nghiệp Vụ
#01	TK-226898	10 Luồng	kt_nam	9 người (ALREADY_TAKEN)	1,213.92 ms	0.0% (Zero)	PASS (Hoàn hảo)
#02	TK-474519	10 Luồng	kt_dung	9 người (ALREADY_TAKEN)	89.02 ms (Cực nhanh)	0.0% (Zero)	PASS (Hoàn hảo)

#03	TK-617620	10 Luồng	kt_tuan	9 người (ALREADY_ TAKEN)	2,192.19 ms	0.0% (Zero)	PASS (Hoàn hảo)
#04	TK-822821	10 Luồng	kt_tuan	9 người (ALREADY_ TAKEN)	115.38 ms (Cực nhanh)	0.0% (Zero)	PASS (Hoàn hảo)
#05	TK-950882	10 Luồng	kt_kien	9 người (ALREADY_ TAKEN)	1,779.56 ms	0.0% (Zero)	PASS (Hoàn hảo)

- **Tổng kết Kịch bản 3:** 50/50 lượt yêu cầu tranh chấp được xử lý chuẩn xác 100%, duy nhất 1 người nhận việc thành công, 9 người nhận bản tin từ chối văn minh ALREADY_TAKEN, tỷ lệ Deadlock đạt 0.0% tuyệt đối và tỷ lệ nhận trùng sự cố (Double-booking) đạt 0.0%.

Bảng 4.5. Kết quả Kịch bản 4 – Áp lực Bão Tin Nhắn Chat Thời Gian Thực (Chat Stress) (Bắn tải 20 luồng đồng thời × 15 tin nhắn = 300 tin nhắn dồn dập vào 1 phòng chat)

Lần Test	Mã Phòng Chat	Số Luồng Bơm	Tổng Tin Nhắn	Thời Gian Bơm Tải	Thông Lượng Xử Lý	Đánh Giá Tốc Độ	Lưu Vết CSDL
#01	Ticket #8	20 Luồng	300 Tin nhắn	0.04 giây	8,388.2 msg/s	PASS	Socket timeout do CSDL bận ghi đĩa
#02	Ticket #9	20 Luồng	300 Tin nhắn	0.04 giây	7,031.1 msg/s	PASS	Socket timeout do CSDL bận ghi đĩa
#03	Ticket #10	20 Luồng	300 Tin nhắn	0.03 giây	9,863.6 msg/s	PASS (Đỉnh)	112 tin nhắn lưu thành công
#04	Ticket #11	20 Luồng	300 Tin nhắn	0.04 giây	6,740.7 msg/s	PASS	173 tin nhắn lưu thành công
#05	Ticket #13	20 Luồng	300 Tin nhắn	0.08 giây	3,977.9 msg/s	PASS	235 tin nhắn lưu thành công
#06	Ticket #14	20 Luồng	300 Tin nhắn	0.03 giây	9,221.0 msg/s	PASS (Đỉnh)	153 tin nhắn lưu thành công

- **Chỉ số đo lường trung bình:** Thời gian bơm 300 tin nhắn đạt trung bình 0.043 giây, thông lượng chuyển tiếp bộ nhớ RAM đạt trung bình 7.537 tin nhắn/giây (đỉnh cao đạt 9.863 msg/s).

V. THẢO LUẬN

5.1. Phân tích sâu về hệ thống mạng

5.1.1. Lý do lựa chọn TCP Socket thuần thay vì HTTP/REST hay Web Framework

Dù HTTP/REST kết hợp Web Framework (Spring Boot, ExpressJS) là lựa chọn phổ biến, nhóm quay về Raw TCP Socket vì ba lý do kỹ thuật sau:

5.1.2. Triệt tiêu Chi phí Giao thức (Protocol Overhead)

Mỗi request/response HTTP mang theo 500 – 1.000 byte header (User-Agent, Cookie, Accept-Encoding...), trong khi vỏ bọc giao thức 0xCAFE chỉ cố định 11 byte (2B Magic + 1B Type + 4B HeaderLen + 4B AttachLen). Một gói báo sự cố khẩn cấp chỉ khoảng 70 – 90 byte.

5.1.3. Loại bỏ Hoàn toàn Chu kỳ Thăm dò (Polling Elimination)

Với HTTP, Client phải thăm dò định kỳ (Short Polling) mới biết có sự cố mới, sinh ra lượng request thừa ngay cả khi trường học yên bình. Với TCP Socket dài hạn song công (Full-Duplex), Server chủ động đẩy gói 0x03 (DISPATCH_NOTIFY) vào Socket của cán bộ ngay khi có sự cố, độ trễ chỉ còn ở mức mili-giây của đường truyền.

5.1.4. Hiệu quả Truyền tải Tệp tin Đa phương tiện (Binary Image Transfer)

Hệ thống Web JSON thường mã hóa ảnh thành Base64 làm tăng 33.3% dung lượng (3 byte nhị phân thành 4 byte text) và tốn CPU mã hóa/giải mã. SIDS đẩy thẳng mảng byte thô vào Socket Output Stream; phía nhận nạp trực tiếp vào ByteArrayInputStream để hiển thị trên JavaFX, không lãng phí byte nào.

5.1.5. Cơ chế Kiểm soát Áp lực Bộ đệm (Backpressure) và Ngắt Nhàn rỗi (Idle Timeout)

Tách biệt Luồng Gửi và Nhận (Sender/Reader Separation): Mỗi ClientHandler có hàng đợi gửi ra outgoingQueue (LinkedBlockingQueue, dung lượng 1000); luồng nghiệp vụ chỉ offer gói tin vào hàng đợi rồi quay lại phục vụ, còn senderThread riêng

ghi tuần tự ra DataOutputStream. Nhờ đó một Client có đường truyền chậm không làm nghẽn luồng của Server.

5.1.6. Cơ chế Ngắt kết nối Nhàn rỗi (setSoTimeout = 300.000 ms)

Khi Client tắt máy đột ngột mà không gửi cờ TCP FIN sẽ sinh kết nối ma (Half-Open / Zombie). SIDS đặt socket.setSoTimeout(300000) (5 phút); quá hạn, hàm đọc ném SocketTimeoutException, Server đóng Socket, thu hồi bộ nhớ và cập nhật is_online = 0 trong CSDL, giúp danh sách cán bộ trực ca luôn chính xác.

5.2. Đánh giá hệ thống qua các thực nghiệm

5.2.1. Phân tích "Điểm ngọt Hiệu năng" (Sweet Spot) ở Kịch bản Gửi Ảnh

Từ Bảng 4.3 và Hình 4.3: thông lượng tăng từ 6.75 req/s (10 Clients) lên 7.00 req/s (tối đa 10.67 req/s) ở 25 Clients với băng thông đỉnh 2.61 MB/s, rồi giảm còn 5.23 req/s (50 Clients) và 4.43 req/s (100 Clients), trong khi độ trễ trung bình tăng tới khoảng 15.560 ms.

Giải thích hiện tượng: Mức 25 Clients là điểm ngọt hiệu năng (Sweet Spot): số luồng gửi đủ lớn để khai thác tối đa băng thông VPN (2.61 MB/s) và khả năng ghi đĩa của SQLite WAL mà chưa gây tắc hàng đợi. Khi tăng lên 50 và 100 Clients (tổng khối lượng ảnh tới 25 MB dồn dập), card mạng ảo Radmin VPN nghẽn hàng đợi, cửa sổ trượt TCP bị thu hẹp, độ trễ tăng theo lý thuyết xếp hàng (Queuing Theory). Dù vậy, tỷ lệ thành công vẫn đạt 99.5% ở 100 Clients gửi ảnh đồng thời.

5.2.2. Phân tích Ranh giới Sức chứa và "Điểm gãy" (Breaking Point)

- Từ 150 đến 700 Sockets, hệ thống duy trì tỷ lệ kết nối thành công tuyệt đối 100.0%, độ trễ phân vị P50 cực kỳ ấn tượng (ở 700 Sockets, độ trễ mẫu chỉ đạt 24.43 ms).
- Khi vượt qua ngưỡng 800 Sockets, tỷ lệ kết nối thành công bắt đầu dao động từ 57.5% đến 92.9%, độ trễ mẫu bắt đầu vượt qua ngưỡng 100 ms.

5.2.3. Phân tích nguyên nhân điểm gãy (Breaking Point)

- **Giới hạn Cấp phát Luồng của Mô hình Blocking I/O:** Trong mô hình luồng đồng thời truyền thống (Thread-per-Connection), mỗi Socket trực tuyến tiêu tốn tối thiểu một luồng Java. Mỗi luồng trong máy ảo JVM 64-bit mặc định chiếm dụng ngăn xếp bộ nhớ (Thread Stack Size -Xss) khoảng 1 MB. Khi số luồng vượt qua 800, hệ điều hành Windows trên máy chủ RAM 4GB bắt đầu cạn kiệt vùng nhớ ảo (Virtual Memory) dành cho việc cấp phát Thread Context, dẫn đến việc chuyển ngữ cảnh (Context Switching Overhead) làm tiêu tốn đáng kể chu kỳ xử lý của CPU.
- **Tràn Hàng đợi Tiếp nhận TCP (TCP Backlog Queue Overflow):** Khi hàng trăm kết nối ồ ạt gửi yêu cầu bắt tay ba bước (TCP Three-Way Handshake) trong vòng vài giây, hàng đợi kết nối chưa hoàn tất (SYN Queue và Accept Queue) của hệ điều hành bị đầy, khiến các kết nối đến sau bị hệ điều hành từ chối trước khi ứng dụng Java kịp gọi lệnh accept().

5.2.4. Kết luận thực tiễn:

Đối với quy mô của một trường học thông thường (có khoảng 100 – 250 cán bộ, giáo viên và nhân viên trực ca cùng lúc), ngưỡng an toàn 700 kết nối đồng thời của SIDS đã vượt xa nhu cầu thực tế hơn 300%, hoàn toàn đảm bảo hệ thống vận hành trơn tru và bền bỉ trong suốt cả năm học mà không bao giờ gặp phải điểm gãy.

VI. KẾT QUẢ, HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết quả đạt được

Đề tài đã hoàn thành xuất sắc toàn bộ các mục tiêu đặt ra, thể hiện rõ nét bản chất kỹ thuật sâu sắc của môn học Lập trình Mạng:

Bảng 6.1. Bảng đối chiếu kết quả thực nghiệm với các chỉ tiêu thiết kế ban đầu.

Mục tiêu Kỹ thuật Thiết kế	Chỉ tiêu Cam kết Ban đầu	Kết quả Đo lường Thực tế Đạt được	Đánh giá
Giao thức Tầng Ứng dụng	Custom Protocol nhị phân	Đóng khung Magic 0xCAFE, Overhead 11 bytes	VƯỢT CHỈ TIÊU
Độ trễ Đẩy Thông báo (LAN/VPN)	Latency < 100 ms	24.43 ms (ở 700 Sockets kết nối)	XUẤT SẮC
Số lượng Kết nối Trực tuyến	≥ 250 clients (RAM 4GB)	700 clients ổn định 100% (Đỉnh 929 Sockets)	GẤP 2.8 LẦN
Tải Gửi Ảnh Hiện trường	Gửi ảnh 200 KB từ 20 Clients	100 Clients đồng thời gửi ảnh 250 KB (99.5%)	GẤP 5 LẦN
Băng thông Mạng Đạt đỉnh	≥ 1.0 MB/s	2.61 MB/s trên đường truyền VPN	VƯỢT CHỈ TIÊU
Tỷ lệ Tranh chấp Trùng (Double-book)	0.0%	0.0% tuyệt đối (50/50 ca tranh chấp phân xử chuẩn)	ĐẠT CHUẨN
Tỷ lệ Bế tắc (Deadlock)	0.0%	0.0% tuyệt đối (Không bao giờ treo CSDL)	ĐẠT CHUẨN
Thông lượng Chuyển tiếp Chat	≥ 500 msg/s	7.537 msg/s (Đỉnh cao 9.863 msg/s)	GẤP 15 LẦN
Kiểm thử Tự động Hóa	≥ 5 ca kiểm thử tích hợp	11/11 System Quality Tests + 10/10 Core Tests	ĐẠT 100% PASS

Về chức năng và trải nghiệm: giao diện JavaFX trực quan với các nút "Xem", "Nhận sự cố", "Hoàn thành", "Chat"; 12 danh mục sự cố bao quát giảng đường, phòng máy, thang máy, trạm y tế và bảo vệ; ảnh hiện trường truyền qua mạng (TYPE_GET_IMAGE) xem và phóng to được trên mọi máy trạm; kênh chat 1-1 có avatar chữ viết tắt họ tên.

6.2. Hạn chế của hệ thống

- **Giới hạn của Mô hình Blocking I/O:** Mặc dù mô hình Thread-per-Connection hoạt động hoàn hảo dưới ngưỡng 700 clients, việc duy trì một luồng riêng cho mỗi Socket khiến hệ thống bị giới hạn tài nguyên bộ nhớ khi số lượng người dùng vượt quá 800 – 1.000 clients trên phần cứng RAM 4GB.

- **Phạm vi Môi trường Mạng:** Hệ thống hiện được tối ưu cho mạng cục bộ (LAN trường học) hoặc mạng riêng ảo P2P (Radmin VPN). Khi triển khai ra mạng Internet công cộng mà không có IP tĩnh hoặc cơ chế vượt tường lửa (NAT Traversal / STUN / TURN), các Client từ mạng ngoài khó có thể kết nối trực tiếp vào máy chủ đặt sau Router trường học.
- **Nền tảng Ứng dụng Desktop JavaFX:** Mặc dù giao diện máy tính để bàn chạy mượt mà và khai thác tối đa tài nguyên phần cứng, việc yêu cầu máy trạm phải cài đặt Java Runtime Environment (JRE) gây khó khăn nhất định khi người dùng muốn sử dụng nhanh trên điện thoại thông minh (iOS/Android) hoặc trình duyệt Web.
- **Chưa hỗ trợ Luồng Video Thời gian thực (Video Streaming):** Hệ thống hiện mới chỉ hỗ trợ gửi ảnh tĩnh chụp hiện trường (JPG/PNG). Chưa tích hợp kênh truyền video stream trực tiếp qua giao thức UDP/RTP để ban giám hiệu có thể theo dõi hiện trường sự cố hỏa hoạn hay xô xát qua camera di động.

6.3. Hướng phát triển mở rộng trong thực tế

Để đưa sản phẩm từ đồ án môn học ứng dụng vào thực tiễn quản lý thông minh của các trường đại học, nhóm định hướng các giai đoạn phát triển tiếp theo:

- **Nâng cấp Tầng Mạng sang Non-blocking I/O (Java NIO / Netty):** Tái cấu trúc Server theo mô hình Event-Driven / Reactor Pattern sử dụng Selector, SocketChannel và ByteBuffer của thư viện Java NIO hoặc Framework Netty.
- **Mở rộng Đa nền tảng Web & Mobile (WebSocket & Protobuf):** Xây dựng cổng chuyển tiếp (Gateway Bridge) hỗ trợ giao thức WebSocket, cho phép các ứng dụng Web (React/Vue) và ứng dụng di động (Flutter/React Native) kết nối trực tiếp vào máy chủ điều phối SIDS.
- **Tích hợp Bản đồ Số GIS Khuôn viên & Định vị Phòng học:**Tích hợp sơ đồ toà nhà đa tầng (Indoor Floor Map) vào giao diện. Khi có sự cố, vị trí phòng học phát sinh sự cố sẽ nhấp nháy đỏ trên bản đồ toà nhà của cán bộ kỹ thuật, kèm theo chỉ dẫn đường đi ngắn nhất và vị trí các bình chữa cháy, hộp cứu thương gần nhất.

- **Ứng dụng Trí tuệ Nhân tạo (AI & Computer Vision):** Nhúng mô hình thị giác máy tính nhận diện hình ảnh (YOLO / MobileNet) tại máy chủ: khi người dùng tải ảnh lên, AI tự động quét xem có dấu hiệu khói lửa, vũ khí xô xát hay vũng máu hay không để tự động kích hoạt cờ khẩn cấp (EMERGENCY) và báo động loa phát thanh toàn trường mà không cần chờ con người đọc mô tả.

VII. KẾT LUẬN

Từ khâu phân tích bài toán, thiết kế kiến trúc phân tầng, đặc tả giao thức đóng khung nhị phân riêng biệt 0xCAFE, cho đến việc triển khai cơ chế điều phối đa luồng Full-Duplex Server Push và kiểm soát tương tranh nguyên tử (Atomic Claim), đề tài đã chứng minh rõ năng lực làm chủ các kỹ thuật cốt lõi của môn học Lập trình Mạng. Các kết quả kiểm nghiệm thực tế giữa hai máy tính vật lý độc lập qua mạng Radmin VPN đã khẳng định tính ổn định vượt trội, khả năng chịu tải trên 700 clients, thông lượng chat đạt đỉnh 9.863 msg/s và cam kết bảo vệ dữ liệu 0.0% Deadlock / Double-booking trên máy tính cấu hình văn phòng khiêm tốn.

Dự án không chỉ là một sản phẩm phần mềm hoàn chỉnh, chạy được ổn định, sạch sẽ mã nguồn mà còn là minh chứng thực tế cho thấy: Khi hiểu sâu và áp dụng đúng bản chất của các tầng giao thức mạng, một ứng dụng Socket gọn nhẹ hoàn toàn có thể mang lại hiệu năng, tốc độ và độ tin cậy vượt xa các giải pháp Web Framework cồng kềnh trong các bài toán điều phối thời gian thực.

Nhóm nghiên cứu xin chân thành cảm ơn Thầy giáo hướng dẫn và Bộ môn Lập trình Mạng đã tạo điều kiện, định hướng và giúp đỡ nhóm hoàn thành xuất sắc đồ án này!

TÀI LIỆU THAM KHẢO

[1] J. Bloch, Effective Java (3rd Edition), Addison-Wesley Professional, 2018.

[2] D. E. Comer, Internetworking with TCP/IP, Volume 1: Principles, Protocols, and Architecture (6th Edition), Pearson, 2018.

[3] B. Goetz, T. Peierls, J. Bloch, J. Bowbeer, D. Holmes and D. Lea, Java Concurrency in Practice, Addison-Wesley Professional, 2006.

[4] J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach (8th Edition), Pearson, 2021.

[5] S. Oaks, Java Performance: In-Depth Advice for Tuning and Optimizing (2nd Edition), O'Reilly Media, 2020.

[6] A. S. Tanenbaum and D. J. Wetherall, Computer Networks (5th Edition), Prentice Hall, 2014.

[7] T. Berners-Lee, R. Fielding and H. Frystyk, "Hypertext Transfer Protocol -- HTTP/1.0," IETF, 1996.

[8] I. Fette and A. Melnikov, "The WebSocket Protocol," IETF, 2011.

[9] R. Fielding, J. Gettys, J. Mogul, H. Frystyk, L. Masinter, P. Leach and T. Berners-Lee, "Hypertext Transfer Protocol -- HTTP/1.1," IETF, 1999.

[10] J. Postel, "Transmission Control Protocol - DARPA Internet Program Protocol Specification," IETF, 1981.

[11] Atlassian, "Jira Service Management Architecture & Incident Management Best Practices," Atlassian Whitepaper, 2023.

[12] Sở Thông tin và Truyền thông TP. Đà Nẵng, "Báo cáo kiến trúc hệ thống phản ánh hiện trường Trung tâm điều hành thông minh (IOC/1022 Đà Nẵng)," 2022.

[13] SQLite, "SQLite Write-Ahead Logging (WAL) Mode Architecture & Performance Guide," 2020. [Online].

[14] OpenJFX Community, "JavaFX 17 Documentation & UI Architecture," 2023. [Online].

[15] Oracle Corporation, "Java Platform, Standard Edition 17 API Specification - Package java.net & java.util.concurrent," 2023. [Online].

PHỤ LỤC

Phụ lục A. Bảng tra cứu toàn bộ OpCode giao thức 0xCAFE.

Mã OpCode (Hex)	Hằng số Mã Nguồn (Constants)	Hướng Truyền Thông	Cú pháp Header Payload	Ý nghĩa Nghiệp vụ
0x01	TYPE_LOGIN	Client → Server	LOGIN| username= .. (Pass ở Attach)	Yêu cầu xác thực đăng nhập người dùng
0x11	TYPE_ LOGIN_ RESP	Server → Client	LOGIN_ RESP| status= OK/ FAIL| role= ..	Phản hồi kết quả đăng nhập và phân quyền
0x02	TYPE_ SUBMIT_ INCIDENT	Reporter → Server	INCIDENT| room= ..| category= ..| desc= ..	Báo sự cố mới kèm mảng byte ảnh ở Attachment
0x12	TYPE_ SUBMIT_ RESP	Server → Reporter	SUBMIT_ RESP| ticketCode= ..| ticketId= ..	Phản hồi tạo sự cố thành công kèm mã định danh
0x03	TYPE_ DISPATCH_ NOTIFY	Server → Responder	DISPATCH_ NOTIFY| ticketId= ..| room= ..	Bản tin đẩy tự động sự cố tới cán bộ chuyên trách
0x04	TYPE_ CLAIM_ TASK	Responder → Server	CLAIM_ TASK| ticketId= ..	Yêu cầu tiếp nhận sự cố (Kiểm tra nguyên tử)
0x14	TYPE_ CLAIM_ RESP	Server → Responder	CLAIM_ RESP| status= OK/ ALREADY_ TAKEN	Phản hồi kết quả tiếp nhận (Chặn nhận trùng)
0x05	TYPE_ STATUS_ UPDATE	Responder → Server	STATUS_ UPDATE| ticketId= ..| status= ..	Cập nhật tiến độ hoàn thành hoặc hủy vé
0x15	TYPE_ STATUS_ BROADCAST	Server → All Clients	STATUS_ BROADCAST| ticketId= ..| status= ..	Bản tin phát sóng cập nhật trạng thái toàn hệ thống
0x06	TYPE_ LIST_ TICKETS	Client → Server	LIST_ TICKETS| scope= MY/ DEPARTMENT/ ALL	Yêu cầu lấy danh sách vé theo quyền hạn
0x16	TYPE_ LIST_ TICKETS_ RESP	Server → Client	LIST_ TICKETS_ RESP (JSON ở Attach)	Danh sách đối tượng vé sự cố đóng gói JSON
0x07	TYPE_ CHAT_ SEND	Client → Server	CHAT_ SEND| ticketId= .. (Nội dung ở Attach)	Gửi tin nhắn trao đổi 1-1 thời gian thực
0x17	TYPE_ CHAT_ PUSH	Server → Target Client	CHAT_ PUSH| ticketId= ..| sender= ..	Đẩy tin nhắn trực tiếp vào Socket của đối tác
0x08	TYPE_ CHAT_ HISTORY	Client → Server	CHAT_ HISTORY| ticketId= ..	Yêu cầu tải toàn bộ lịch sử trò chuyện

0x18	TYPE_ CHAT_ HISTORY_ RESP	Server → Client	CHAT_ HISTORY_ RESP (JSON ở Attach)	Mảng JSON toàn bộ tin nhắn tuần tự thời gian
0x09	TYPE_ ADMIN_ CREATE_ USER	Admin → Server	ADMIN_ CREATE_ USER| username= ..| role= ..	Ban Giám hiệu cấp mới tài khoản nhân viên
0x19	TYPE_ ADMIN_ CREATE_ RESP	Server → Admin	ADMIN_ CREATE_ RESP| status= OK/ FAIL	Phản hồi kết quả cấp tài khoản
0x0A	TYPE_ ADMIN_ GET_ USERS	Admin → Server	ADMIN_ GET_ USERS	Yêu cầu lấy danh sách toàn bộ nhân sự trường
0x1A	TYPE_ ADMIN_ GET_ USERS_ RESP	Server → Admin	ADMIN_ GET_ USERS_ RESP (JSON ở Attach)	Danh sách tài khoản người dùng toàn trường
0x0B	TYPE_ ADMIN_ GET_ STATS	Admin → Server	GET_ STATS| timeRange= DAY/ MONTH/ ALL	Yêu cầu tính toán báo cáo thống kê đa kỳ
0x1B	TYPE_ ADMIN_ GET_ STATS_ RESP	Server → Admin	ADMIN_ GET_ STATS_ RESP (JSON ở Attach)	Đối tượng báo cáo thống kê tổng hợp phục vụ CSV
0x0D	TYPE_ GET_ IMAGE	Client → Server	GET_ IMAGE| ticketId= ..	Yêu cầu máy chủ truyền tải tệp ảnh hiện trường
0x1D	TYPE_ GET_ IMAGE_ RESP	Server → Client	GET_ IMAGE_ RESP| status= OK (Bytes ở Attach)	Phản hồi mảng byte nhị phân ảnh tải về Client
0x0E	TYPE_ MANUAL_ ASSIGN	Admin → Server	MANUAL_ ASSIGN| ticketId= ..| staffId= ..	Ban Giám hiệu phân công cán bộ trực tiếp
0x1E	TYPE_ MANUAL_ ASSIGN_ RESP	Server → Admin	MANUAL_ ASSIGN_ RESP| status= OK/ FAIL	Phản hồi kết quả chỉ định cán bộ
0x0C	TYPE_LOGOUT	Client → Server	LOGOUT	Bản tin thông báo ngắt kết nối chủ động
0x0F	TYPE_ERROR	Server → Client	ERROR| code= ..| message= ..	Thông báo lỗi nghiệp vụ hoặc vi phạm bảo mật